

TRABAJO PRÁCTICO INTEGRADOR

Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas

Institución	Universidad Tecnológica Nacional
Carrera	Tecnicatura Universitaria en Programación
Materia	Programación 1
Año / ciclo	2026
Fecha de entrega	17 de junio de 2026

Integrante

Apellido y nombre	Legajo / DNI	Correo electrónico
Lautaro Borges Licciardi	44476186	lautibor@gmail.com

Repositorio GitHub

https://github.com/Lau-bor/TPI_Paises_GitHub

Índice de contenidos

1. Objetivo del trabajo	2
2. Marco teórico e investigación	3
3. Decisiones técnicas y arquitectura	6
4. Diagrama de flujo del sistema	7
5. Ejemplos de ejecución en consola	8
6. Dificultades y conclusiones	9
7. Bibliografía / webgrafía y links del proyecto	10

1. Objetivo del trabajo

El objetivo del Trabajo Práctico Integrador es desarrollar una aplicación de consola en Python que permita gestionar información sobre países. El sistema trabaja con un archivo CSV como fuente de datos y aplica listas, diccionarios, funciones, condicionales, bucles, filtros, ordenamientos y cálculos estadísticos básicos.

La propuesta busca integrar los contenidos centrales de Programación 1 en un caso práctico: cargar datos, validarlos, consultarlos, modificarlos y obtener indicadores útiles del dataset.

2. Marco teórico e investigación

En esta sección se explican los conceptos aplicados en el desarrollo del sistema y su relación con el código implementado.

2.1 Listas en Python

Una lista es una estructura de datos ordenada y mutable que permite almacenar varios elementos bajo un mismo nombre. En Python se define utilizando corchetes y puede contener datos de distintos tipos. En este proyecto, la variable principal `países` es una lista que almacena todos los registros cargados desde el archivo CSV.

La lista permite recorrer los países con bucles `for`, agregar nuevos elementos con `append()` y calcular resultados a partir de todos los registros disponibles.

```
países = []

país = {
    "nombre": "Argentina",
    "población": 45376763,
    "superficie": 2780400,
    "continente": "América"
}

países.append(país)
```

2.2 Diccionarios en Python

Un diccionario es una estructura de pares clave-valor. Permite acceder a cada dato mediante una clave con significado semántico, por ejemplo `nombre`, `población`, `superficie` o `continente`. Esta representación evita depender de posiciones numéricas y mejora la legibilidad del código.

```
país = {
    "nombre": "Brasil",
    "población": 213993437,
    "superficie": 8515767,
    "continente": "América"
}

print(país["nombre"])
país["población"] = 214000000
```

2.3 Funciones y modularización

Una función es un bloque de código reutilizable que realiza una tarea específica. En el proyecto se aplico el criterio una función = una responsabilidad, separando el código en operaciones claras: lectura del CSV, guardado, alta, actualización, búsqueda, filtros, ordenamientos, estadísticas, listado y validaciones.

Función	Responsabilidad
cargar_paises(ruta)	Lee el CSV, valida filas y devuelve una lista de diccionarios.
guardar_paises(países, ruta)	Guarda la lista actualizada en el archivo CSV.
agregar_pais(países)	Solicita, valida y agrega un nuevo país.
actualizar_pais(países)	Actualiza población y superficie de un país existente.
buscar_pais(países)	Busca coincidencias parciales o exactas por nombre.
filtrar_paises(países)	Permite filtrar por continente, población o superficie.
ordenar_paises(países)	Ordena por nombre, población o superficie.
mostrar_estadisticas(países)	Calcula indicadores generales del dataset.
mostrar_tabla(países)	Muestra resultados en formato tabular.
pedir_entero_positivo(mensaje)	Valida entradas numericas positivas.

2.4 Estructuras condicionales y repetitivas

Las estructuras condicionales permiten tomar decisiones mediante if, elif y else. En el sistema se utilizan para validar opciones del menú, detectar datos inválidos y determinar que función debe ejecutarse. Las estructuras repetitivas permiten recorrer colecciones o repetir una acción hasta obtener una entrada válida.

```
while True:
    opción = input("Seleccione una opción: ").strip()

    if opción == "1":
        agregar_pais(países)
    elif opción == "0":
        break
    else:
        print("[ERROR] Opción no valida.")
```

2.5 Ordenamientos

Ordenar datos consiste en reorganizar una colección según un criterio. En Python se puede utilizar `sorted()`, que devuelve una nueva lista ordenada sin modificar la lista original. En este proyecto el usuario puede ordenar por nombre, población o superficie, y elegir orden ascendente o descendente.

Para mantener el código claro, se definieron funciones auxiliares como `obtener_nombre()`, `obtener_poblacion()` y `obtener_superficie()`, que se usan como criterio de ordenamiento.

```
def obtener_poblacion(país):  
    return país["población"]  
  
ordenados = sorted(países, key=obtener_poblacion, reverse=True)
```

2.6 Estadísticas básicas

Las estadísticas descriptivas permiten resumir un conjunto de datos. El sistema calcula país con mayor población, país con menor población, promedio de población, promedio de superficie y cantidad de países por continente.

```
mayor_población = max(países, key=obtener_poblacion)  
menor_población = min(países, key=obtener_poblacion)  
  
promedio_población = total_población / cantidad_países
```

2.7 Archivos CSV

CSV significa Comma-Separated Values. Es un formato de texto plano utilizado para guardar datos tabulares. En este proyecto se utiliza el módulo `csv` de la biblioteca estándar de Python. La lectura se realiza con `csv.DictReader` y la escritura con `csv.DictWriter`.

El sistema valida que el archivo tenga encabezados correctos, que no existan campos vacíos, que población y superficie sean enteros positivos y que no se repitan países.

3. Decisiones técnicas y arquitectura

3.1 Estructura del proyecto

El proyecto se organizo en un unico archivo fuente `países.py`, separado en bloques lógicos mediante comentarios. Esta decisión simplifica la entrega y ejecución del sistema, ya que la escala del trabajo no requiere dividir el código en multiples modulos.

Archivo	Contenido
<code>países.py</code>	Código fuente principal: carga CSV, menú, funciones, validaciones y estadísticas.
<code>países.csv</code>	Dataset base de países con nombre, población, superficie y continente.
<code>README.md</code>	Descripcion del proyecto, instrucciones de uso, ejemplos y links de entrega.
<code>informe_tpi.pdf</code>	Documentacion academica y tecnica del trabajo.

3.2 Estructura de datos

La estructura principal es una lista de diccionarios. Cada diccionario representa un país y contiene las claves nombre, población, superficie y continente. Esta elección es adecuada porque permite recorrer todos los países y, al mismo tiempo, acceder a cada atributo con una clave descriptiva.

3.3 Metodología utilizada

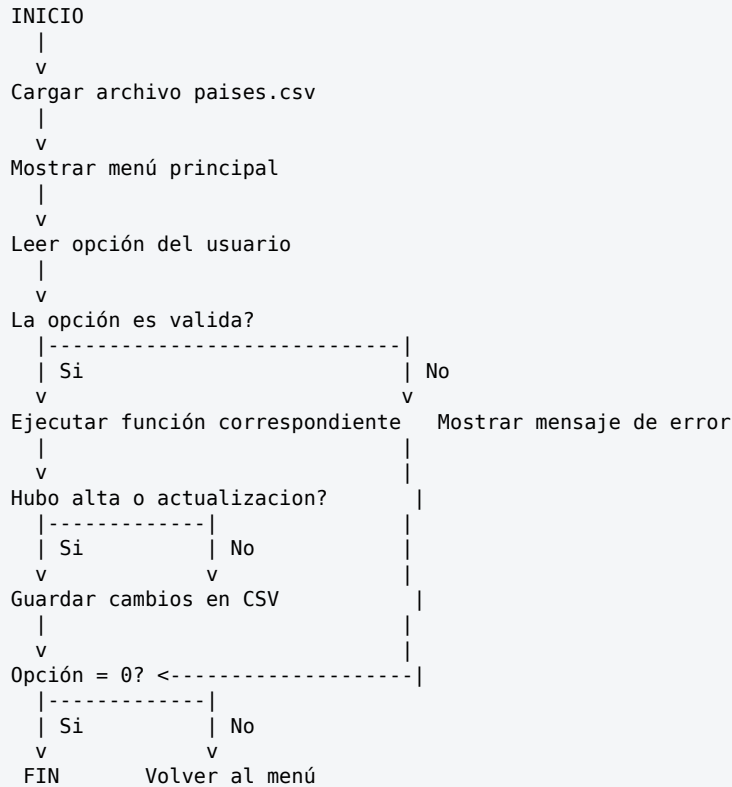
- Analisis del enunciado y de los requerimientos minimos del sistema.
- Definicion del formato de datos y del archivo CSV base.
- Implementacion progresiva de funcionalidades mediante funciones.
- Validación de entradas y manejo basico de errores.
- Pruebas manuales de búsqueda, filtros, ordenamientos, estadísticas y persistencia.
- Documentacion del proyecto en README e informe academico.

3.4 Repositorio GitHub

El repositorio público del proyecto se encuentra en: https://github.com/Lau-bor/TPI_Paises_GitHub

4. Diagrama de flujo del sistema

El siguiente esquema representa el flujo general de ejecución del programa desde la carga inicial de datos hasta la finalización del sistema.



4.1 Módulos lógicos del programa

Modulo lógico	Funciones principales
Datos CSV	cargar_paises, guardar_paises
Alta y actualizacion	agregar_pais, actualizar_pais
Búsqueda	buscar_pais, buscar_indice_exacto
Filtros	filtrar_paises, filtrar_por_continente, filtrar_por_rango
Ordenamiento	ordenar_paises, obtener_nombre, obtener_poblacion, obtener_superficie
Estadísticas	mostrar_estadisticas
Interfaz de consola	mostrar_menu, main, listar_paises, mostrar_tabla

5. Ejemplos de ejecución en consola

A continuación se presentan salidas representativas del programa. Los resultados fueron corregidos para coincidir con el dataset base utilizado en el proyecto.

5.1 Menú principal

```
=====
SISTEMA DE GESTION DE PAISES
=====
1. Agregar país
2. Actualizar datos de un país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Listar todos los países
0. Salir
-----
Seleccione una opción:
```

5.2 Estadísticas globales

```
--- Estadísticas globales ---

Total de países registrados : 44

Mayor población -> China (1,444,216,107 hab.)
Menor población -> Fiyi (930,748 hab.)

Promedio de población : 133,842,241 habitantes
Promedio de superficie : 1,827,214 km2

Países por continente:
- África          10 país/es
- América         10 país/es
- Asia            10 país/es
- Europa          10 país/es
- Oceanía         4 país/es
```

5.3 Búsqueda parcial por 'ar'

```
--- Buscar país ---
Ingrese nombre (parcial o exacto): ar

Se encontraron 4 resultado(s):
Nombre          Población      Superficie  Continente
-----
Argentina       45,376,763     2,780,400 km2  América
Arabia Saudita  35,013,414     2,149,690 km2  Asia
Argelia         44,616,624     2,381,741 km2  África
Marruecos       37,457,971     710,850 km2   África
```

5.4 Validación de entrada inválida

```
--- Agregar nuevo país ---
Nombre: Uruguay
Población: abc
[ERROR] Ingrese un número entero válido.
Población: -500
[ERROR] El valor debe ser mayor que cero.
Población: 3473730
Superficie (km2): 176215
Continente: América

[OK] País 'Uruguay' agregado correctamente.
```


6. Dificultades y conclusiones

6.1 Dificultades encontradas

Validación robusta del CSV

Al leer el archivo CSV podían aparecer filas incompletas, campos vacíos o valores no numéricos. Para evitar que el programa se detenga, cada fila se procesa dentro de un bloque try/except y, ante un error, se informa el número de fila y se continúa con las siguientes.

Persistencia inmediata de los cambios

Cuando se agrega o actualiza un país, el sistema llama a `guardar_paises()` inmediatamente. Esto evita que los cambios queden solo en memoria y se pierdan al cerrar el programa.

Búsquedas insensibles a mayúsculas

Para que la búsqueda sea más flexible, se compara el término ingresado por el usuario con los nombres de los países utilizando `lower()`. De este modo, Argentina, argentina o ARGENTINA producen el mismo resultado.

Alíneación de resultados

La tabla de resultados se ajusta al nombre más largo encontrado. Esto permite mantener columnas legibles al listar países con nombres de distinta longitud.

6.2 Conclusiones

- El trabajo permitió integrar estructuras de datos, funciones, archivos y validaciones en una aplicación funcional.
- Las listas y diccionarios resultaron adecuados para representar el dataset de países de manera clara.
- La modularización facilitó el desarrollo, las pruebas y la corrección de errores.
- El manejo de archivos CSV permitió incorporar persistencia sin utilizar bases de datos externas.
- Los filtros, ordenamientos y estadísticas muestran cómo transformar datos en información útil para el usuario.

7. Bibliografía / webgrafía y links del proyecto

7.1 Bibliografía / webgrafía

- Python Software Foundation. (2024). Python 3 Documentation - Built-in Types, csv module, Control Flow. <https://docs.python.org/3/>
- Van Rossum, G. & Drake, F. L. (2023). The Python Language Reference. <https://docs.python.org/3/reference/>
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- McConnell, S. (2004). Code Complete: A Practical Handbook of Software Construction (2nd ed.). Microsoft Press.
- Real Python. (2024). Python's csv module: Reading and Writing CSV Files. <https://realpython.com/python-csv/>
- Real Python. (2024). Sorting Data With Python. <https://realpython.com/python-sort/>

7.2 Links del proyecto

Recurso	Link / ubicacion
Repositorio GitHub	https://github.com/Lau-bor/TPI_Paises_GitHub
Video demostrativo	https://youtu.be/QHzE2zkhyWQ